

Hey guys! Today we are here to solve a new alert on Let's Defend — SOC302: Suspicious Base64 Encoding/Decoding Commands Detected. Let's go like this!



In this writeup, we investigate **Event ID 278** triggered on July 17, 2024, at 12:18 PM. The alert flagged suspicious Base64 encoding/decoding activity on host **Wilburn** (IP: 172.16.17.74), specifically the command:

Python

```
decoded = base64.b64decode(encoded)
```

This is a common technique used by attackers for **data obfuscation**, payload delivery, or hiding malicious scripts during the initial access or execution phase.

We'll walk through the full investigation: alert triage, command line analysis, related brute-force activity from external IP 143.244.44.163, IOC extraction, and defensive recommendations.

Connect with me:

- **LinkedIn:** [Ashvin Acharya](#)
- **GitHub:** [SOC Investigation Repository](#)
- **X / Twitter:** [@OxAshvin](#)

Stay tuned as we break down whether this Base64 activity was part of a successful attack chain or just suspicious noise! 🚀

Alert Details

EventID : 278

Event Time : Jul, 17, 2024, 12:18 PM

Rule : SOC302 — Suspicious Base64 Encoding/Decoding Commands
Detected

Level : Incident Responder

Hostname : Wilburn

Ip Address : 172.16.17.74

Command Line : decoded = base64.b64decode(encoded)

Trigger Reason : Detects suspicious use of Base64 encoding or decoding commands which could be used for data obfuscation.

L1 Note : Minutes before the alert, I saw a Brute Force attempt with different users from the IP 143.244.44.163 towards the system. However, I could not determine whether this attack was successful or not.

Initial Detection: Brute-Force Compromise

The attack began at **06:43 AM**, originating from IP **143.244.44.163**, targeting **SSH (port 22)** on Wilburn. Over a short period, the attacker attempted multiple logins using usernames including `letsdefend`, `fenner`, and `analyst`. After **15 login attempts**, the attacker successfully compromised the `analyst` user account at **06:44 AM**.

33 events (before Jul, 17, 2024, 06:50 AM UTC)

Event	source_address=143.244.44.163	source_port=24650	destination_address=172.16.17.74	destination_port=22	raw_log: {Action Details: "Failed password for invalid user analyst from 143.2..."}
[Jul, 17, 2024, 06:43 AM]	source_address=143.244.44.163	source_port=7508	destination_address=172.16.17.74	destination_port=22	raw_log: {Action Details: "Failed password for invalid user analyst from 143.2..."}
[Jul, 17, 2024, 06:43 AM]	source_address=143.244.44.163	source_port=32873	destination_address=172.16.17.74	destination_port=22	raw_log: {Action Details: "Failed password for analyst from 143.244.44.163 p..."}
[Jul, 17, 2024, 06:43 AM]	source_address=143.244.44.163	source_port=33799	destination_address=172.16.17.74	destination_port=22	raw_log: {Action Details: "Failed password for analyst from 143.244.44.163 p..."}
[Jul, 17, 2024, 06:44 AM]	source_address=143.244.44.163	source_port=39273	destination_address=172.16.17.74	destination_port=22	raw_log: {Action Details: "Accepted password for analyst from 143.244.44.163 p..."}
[Jul, 17, 2024, 06:48 AM]	source_address=172.16.17.74	source_port=0	destination_address=172.16.17.74	destination_port=0	raw_log: {Log Type: "Audit", "Raw Log": "type=EXECVE msg=audit(1721198935.795:2...)"}
[Jul, 17, 2024, 06:48 AM]	source_address=172.16.17.74	source_port=0	destination_address=172.16.17.74	destination_port=0	raw_log: {Log Type: "Audit", "Raw Log": "type=EXECVE msg=audit(1721198899.547:2...)"}

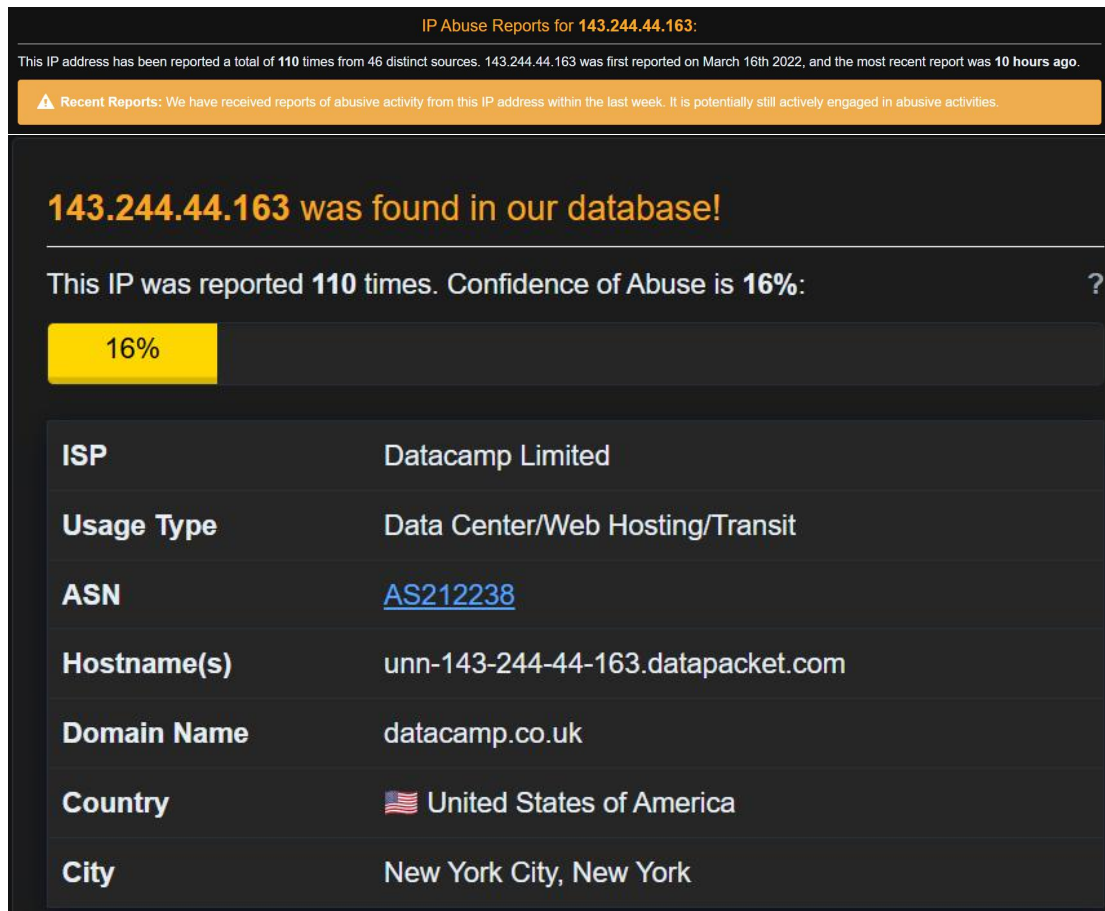
Fig 1. Failed Attempts of SSH to Various Users

Field	Value
type	OS
source_address	143.244.44.163
source_port	39273
destination_address	172.16.17.74
destination_port	22
time	Jul, 17, 2024, 06:44 AM
Raw Log	
Action Details	Accepted password for analyst from 143.244.44.163 port 39273 ssh2

Fig 2. Successful Login to 172.16.17.74 Host through SSH

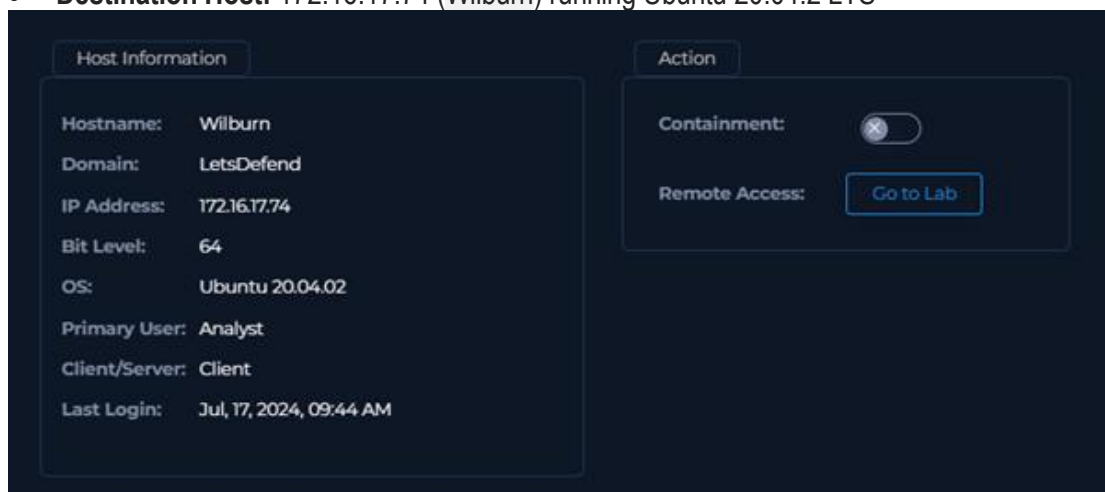
Key details about the attacker's source:

- Source IP: 143.244.44.163 (Datacamp Limited, USA)



Flg 3. Source IP Address Details

- **Abuse History:** Reported 110 times on AbuseIPDB for brute-force, port scanning, and web attacks
- **Destination Host:** 172.16.17.74 (Wilburn) running Ubuntu 20.04.2 LTS



Flg 4. Host Information

The successful compromise gave the attacker interactive access to the host via SSH, initiating a classic **hands-on-keyboard post-compromise scenario**.

Post-Compromise Reconnaissance

Once logged in, the attacker executed a series of commands indicative of systematic reconnaissance and privilege assessment:



```
Jul 17 2024 06:45:37 — dircolors -b
Jul 17 2024 06:45:41 — whoami
Jul 17 2024 06:45:53 — groups
Jul 17 2024 06:46:02 — hostname
Jul 17 2024 06:46:23 — cat /etc/os-release
Jul 17 2024 06:47:23 — uname -a
Jul 17 2024 06:47:30 — cat /etc/group
Jul 17 2024 06:47:32 — clear
Jul 17 2024 06:47:57 — getent passwd
Jul 17 2024 06:48:19 — find / -type f -name *password*
Jul 17 2024 06:48:55 — find / -type f -name *important*
Jul 17 2024 06:49:09 — ls — color=auto
Jul 17 2024 06:49:15 — cat important
Jul 17 2024 06:50:34 — python3 -c 'import base64; with open("/root/Documents/important", "r")
as f: encoded = f.read(); decoded = base64.b64decode(encoded); with
open("/root/Documents/decoded_file.txt", "wb") as out: out.write(decoded)'
Jul 17 2024 06:50:46 — cat decoded_file.txt
```

1. User & Group Enumeration:

- `whoami`, `groups`, `/etc/group`, `getent passwd`
- Purpose: Identify account privileges and potential lateral movement targets

2. System & OS Information Gathering:

- `hostname, cat /etc/os-release, uname -a`
- Purpose: Determine OS version, kernel, and environment for attack planning

3. Filesystem Discovery:

- `find / -type f -name *password*`
- `find / -type f -name *important*`
- Purpose: Search for sensitive credentials and data

4. Targeted File Access:

- Located a file named `important` in `/root/Documents/`
- Renamed (`mv important.txt important`) and viewed via `cat`

5. Obfuscation Bypass:

- Executed Python command to decode Base64 content:

```
python3 -c 'import base64; with open("/root/Documents/important", "r") as f: encoded = f.read();  
decoded = base64.b64decode(encoded); with open("/root/Documents/decoded_file.txt", "wb") as out:  
out.write(decoded)'
```

- Result: Creation of `decoded_file.txt` containing plaintext credentials

The attacker then reviewed the decoded credentials (`cat decoded_file.txt`), confirming **intentional harvesting of sensitive information** rather than casual exploration.

POST-EXPLOITATION COMMAND AUDIT LOG			
Date & Time	Command	What the command does	Why attacker runs it
Jul 17 2024 06:45:37	dircolors -b	Displays shell color configuration	Confirms interactive shell; improves terminal usability
Jul 17 2024 06:45:41	whoami	Shows current logged-in user	Verifies access level and identity
Jul 17 2024 06:45:53	groups	Lists groups of current user	Checks for sudo or privileged group membership
Jul 17 2024 06:46:02	hostname	Displays system hostname	Identifies compromised host for mapping
Jul 17 2024 06:46:23	cat /etc/os-release	Shows Linux distribution and version	Determines OS for exploit/priv-esc selection
Jul 17 2024 06:47:23	uname -a	Displays kernel version and architecture	Identifies kernel vulnerabilities
Jul 17 2024 06:47:30	cat /etc/group	Lists all local groups	Identifies privileged and service groups
Jul 17 2024 06:47:32	clear	Clears terminal output	Hides previous commands / improves readability
Jul 17 2024 06:47:57	getent passwd	Lists all system user accounts	Enumerates users for lateral movement
Jul 17 2024 06:48:19	find / -type f -name *password*	Searches for files named "password"	Hunts for hardcoded or exposed credentials
Jul 17 2024 06:48:55	find / -type f -name *important*	Searches for files named "important"	Looks for sensitive business or credential data
Jul 17 2024 06:49:09	ls --color=auto	Lists directory contents	Confirms presence of discovered files
Jul 17 2024 06:49:15	cat important	Displays file contents	Reads sensitive data found on system
Jul 17 2024 06:50:34	python3 -c 'import base64; with open("/root/Documents/important", "r") as f: encoded = f.read(); decoded = base64.b64decode(encoded); with open("/root/Documents/decoded_file.txt", "wb") as out: out.write(decoded)'	Reads Base64-encoded data from a file and decodes it into a new file	Decodes hidden/obfuscated sensitive data (credentials, configs, or stolen data)
Jul 17 2024 06:50:46	cat decoded_file.txt	Displays contents of decoded file	Views extracted sensitive information

Fig 5. Detailed Command Explanation

Nature of Stolen Data

To determine what type of information was stored in the `/root/Documents/important` and `/root/Documents/decoded_file.txt` files, we accessed the compromised host and reviewed both files, where we found the following details.

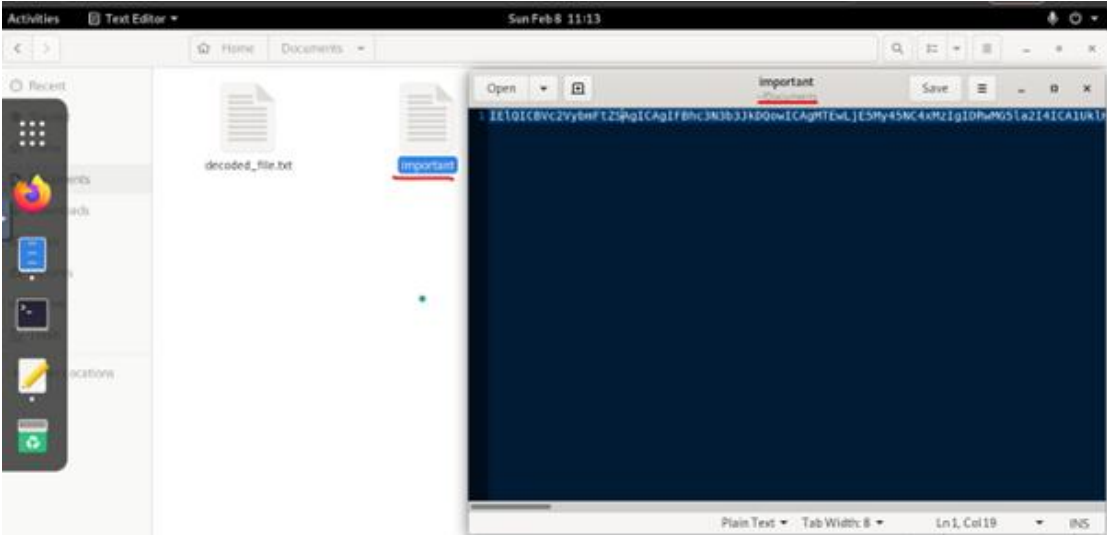


Fig 6. Important File

The decoded file contained a structured table of external IP addresses paired with usernames and passwords:

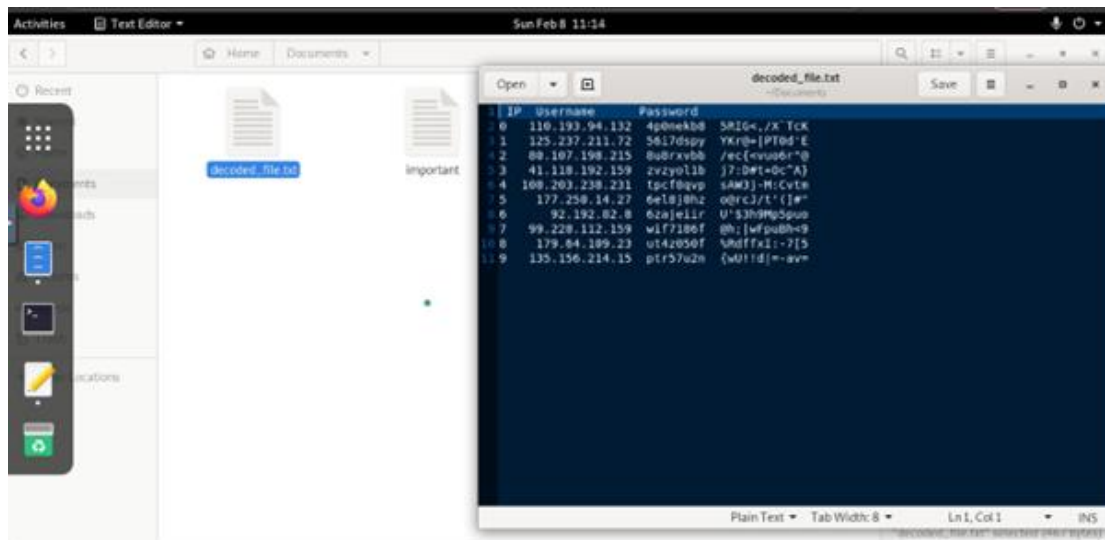


Fig 7. decoded_file.txt File

This clearly indicates the attacker's intent to collect **ready-to-use credentials**, likely for:

- Lateral movement to other hosts
- SSH access to external systems
- Preparation for further compromise or ransomware staging

Attack Timeline (Jul 17, 2024)

1. 06:43 AM — Brute-force attack initiated from 143.244.44.163
2. 06:44 AM — SSH login successful as analyst user
3. 06:45 AM — Interactive session begins; user context verified (whoami, groups)
4. 06:46 AM — System information gathered (hostname, cat /etc/os-release, uname -a)
5. 06:47 AM — User and group enumeration (/etc/group, getent passwd)
6. 06:48 AM — Filesystem search for sensitive files (*password*, *important*)
7. 06:49 AM — File important located and accessed

8. 06:50 AM — Base64 file decoded via Python; `decoded_file.txt` created

9. 06:50 AM- Decoded credentials reviewed (`cat decoded_file.txt`)

Press enter or click to view image in full size



Fig 8. Attack Flow Diagram

Defense Evasion Techniques Observed

The attacker employed **obfuscation and concealment** to evade detection:

- **Obfuscated Files or Information:** Base64 encoding used to hide credentials
- **Deobfuscate/Decode Files or Information:** Python command used to decode data for operational use

No evidence was found of pre-OS boot attacks, reflective code execution, or network tunneling.

Artifacts Collected

- **Files:** `/root/Documents/important` (Base64-encoded credentials), `/root/Documents/decoded_file.txt` (decoded credentials)
- **Commands executed:** System reconnaissance: `whoami`, `groups`, `hostname`, `uname -a`, `/etc/os-release`, User enumeration: `/etc/group`, `getent passwd`, File search: `find / -type f -name *password*`, `find / -type f -name *important*`, Base64 decoding: Python one-liner
- **Processes and Parent:** Shell: `/usr/bin/bash`, Parent process: `/usr/libexec/gnome-terminal-server`

Let's Defend Playbooks Answers

1. Should the device be isolated? — Yes
2. Determine the Scope — No
3. IOC Reputation Check — Yes
4. IOC Analysis — Yes
5. Determine the type of Defense Evasion Techniques Involving Obfuscation and Concealment — Deobfuscate/Decode Files or Information
6. Determine the OS (Operating system) — Linux

Alert Closure Note

This SOC302 alert has been investigated and is now closed.

Classification: True Positive

Reason: The alert correctly identified real malicious activity. An attacker brute-forced SSH from IP 143.244.44.163, successfully logged into host **Wilburn** (172.16.17.74), and then ran multiple reconnaissance commands followed by Base64 decoding to access hidden sensitive data. (**Just for reference - You should prefer your own wording.**)

Why Closing: The attack chain was fully identified, contained, and remediated within the lab environment.

Status: ✓ **Closed** (True Positive - Contained)

Containment and Remediation

To contain this incident and prevent any further damage, the following steps are critical:

1. **Immediately isolate** the compromised host (172.16.17.74) from the network — better to cut it off early than let the attacker continue exploring.
2. Terminate all suspicious SSH sessions originating from the attacker's IP (143.244.44.163).
3. Block the attacker's IP across your firewall, IDS, and IPS rules to slam the door shut.
4. Rotate the password for the compromised **analyst** account right away — and make it a strong one this time.
5. Review and revoke any SSH keys that may have been exfiltrated during the breach.
6. Implement account lockouts for repeated failed SSH login attempts — because nobody should be allowed unlimited guesses at your front door.
7. Enable multi-factor authentication (MFA) for all privileged accounts — it's 2026, this should already be standard practice.

Conclusion

This hands-on exercise with the Let's Defend alert gave us a solid look at how attackers move — from grinding through SSH brute-force attempts all the way to post-compromise reconnaissance and credential decoding. It's one of those scenarios that reminds you just how patient (and persistent) some adversaries can be.

Through this investigation, we picked up a few practical takeaways:

- **Keep a close eye** on repeated failed SSH logins and consider implementing rate-limiting or account lockouts — because letting someone knock on the door a thousand times is never a great security strategy.
- Watch out for suspicious file access and Base64 decoding activity, which often signals someone trying to hide their tracks with a bit of obfuscation.
- Regularly review command histories and process executions — it's one of the best ways to tell the difference between noisy automated tools and a more deliberate human attacker.
- Mapping the full attack timeline helps you quickly connect the dots and get a clearer picture of how the adversary is operating.
- Practicing structured analysis in simulated environments like this is genuinely one of the best ways to sharpen incident response skills.

Overall, this alert reinforced just how important active log monitoring, strong endpoint visibility, and methodical investigation really are. It delivered some solid, real-world-style experience for SOC and IR analysts when it comes to spotting and dissecting credential theft attempts.